

Chapter 4

Higher Inductive Types

Coquand et al. (2018) は、cubical type theory を高次帰納型 (*higher inductive types*) へ拡張する方法を与えた。その鍵が、まさに Section 3.3–Section 3.4 で採った「合成を hcomp と transp に分ける」提示である：高次帰納型では、合成構造を「構成子としての hcomp 」と「構成子ごとに与える transp の κ -簡約」として指定できる。

4.1 A Common Pattern for Higher Inductive Types

CHM の考える高次帰納型の例はすべて共通のパターンに従う。これは cubical type theory における高次帰納型の構文的スキーマの定式化への第一歩とみなせるが、スキーマの正確な定式化とその意味論的対応物は future work とされている。

各高次帰納型 $D(\vec{z} : \vec{P})$ は、(ambient context Γ 上の) パラメータの telescope $\vec{z} : \vec{P}$ と構成子のリスト $\vec{c}$ で指定される^{*1}。各構成子 c は次のデータで指定される：

$$c : (\vec{x} : \vec{A}(\vec{z})) [\vec{i}] D(\vec{z}) \quad \text{境界データ } \varphi(\vec{i}) \mapsto e(\vec{z}, \vec{x}, \vec{i})$$

ここで telescope $\vec{x} : \vec{A}$ は c の引数の型を指定する (propositional truncation のような再帰的高次帰納型では、 D 自身が $\vec{A}$ に現れてよい)。名前 $\vec{i}$ の長さは、 c が導入する立方体の次元を指定する：長さが 0, 1, 2 のとき、 c をそれぞれ point 構成子、path 構成子、square 構成子と呼ぶ。境界データ $\varphi \mapsto e$ は構成子 c の端点を指定する： φ は自由変数が $\vec{i}$ に含まれる $\mathbb{F}$ の元であり、 e は部分元

$$\vec{z} : \vec{P}, \vec{x} : \vec{A}(\vec{z}), \vec{i} : \mathbb{I}, \varphi(\vec{i}) \vdash e(\vec{z}, \vec{x}, \vec{i}) : D(\vec{z})$$

であって、リスト $\vec{c}$ の中で先行する構成子（および後述の hcomp ）のみに言及するものである。

telescope $\vec{z} : \vec{P}$ の各インスタンス $\vec{u} : \vec{P}$ に対して $D(\vec{u})$ は型であり、上のように指定された構成子 c の導入規則は (ambient context を省略して) 次で与えられる：

$$\frac{\vec{v} : \vec{A}(\vec{u}) \quad \vec{r} : \mathbb{I}}{c \vec{v} \vec{r} : D(\vec{u})}$$

さらに $\varphi(\vec{r}) = 1_{\mathbb{F}}$ が成り立つ場合には、判断的等値性

$$c \vec{v} \vec{r} =_{\partial} e(\vec{u}, \vec{v}, \vec{r}) : D(\vec{u})$$

^{*1} 文脈 Γ 上の telescope $x_1 : A_1, \dots, x_n : A_n$ ($\vec{x} : \vec{A}$ と書く) とは、 $\Gamma, \vec{x} : \vec{A}$ が well-formed な文脈となるような (空でもよい) 対象変数宣言のリストである。したがって $\vec{x} : \vec{A}$ は文脈制限 Δ, φ も名前宣言 $i : \mathbb{I}$ も含まない。

が成り立つ (Section 3.6 の ∂ -等値性にあたる)。この判断的等値性のため、関数 $f : (x : D(\vec{u})) \rightarrow P(x)$ を定義するときには、 f がこの等値性を保存すること、すなわち

$$\varphi(\vec{r}) \vdash f(c \vec{v} \vec{r}) = f(e(\vec{u}, \vec{v}, \vec{r})) : P(c \vec{v} \vec{r})$$

が保証されなければならない。この要件は $D(\vec{u})$ の除去規則 (eliminator) の型付け規則で扱われるべきものであるが、その一般的定式化は (「extension types」に類する拡張を要するため) future work とされている。

Remark 36 の見取り図で予告した第三の場合が、まさに高次帰納型である：path 構成子があるため、側面は構成子頭とは限らず、hcomp の κ -簡約は与えられない。そこで homogeneous composition の構造は「構成子として」導入される。といっても、新しい規則は要らない。 $A := D(\vec{u})$ に対する hcomp の型付けと ∂ -等値性は、Definition 29 がそのまま与える ($D(\vec{u})$ は Γ で型付けされ、方向 i に依存しない—homogeneous composition が適用できるのはこのためである。 i は側面 v の中には自由に現れてよい)。高次帰納型に固有なのは、次の 2 点だけである。第一に、この hcomp には κ -簡約を **与えない**。したがって $\text{hcomp}_{D(\vec{u})}^i [\varphi \mapsto v] v_0$ という形の元それ自体が $D(\vec{u})$ の標準形となる (暗黙の構成子とみなしてよい)。第二に、その代償として、 $D(\vec{u})$ からの除去写像には hcomp との可換性を述べる計算規則が付く： $f : D(\vec{u}) \rightarrow C$ に対し $f(\text{hcomp}_{D(\vec{u})}^i [\varphi \mapsto v] v_0) \rightarrow_{\beta} \text{hcomp}_C^i [\varphi \mapsto f(v)] (f(v_0))$ の形の規則である (Section 4.4 の $\mathbb{Z}$ の除去規則を参照。依存除去の一般形は上述のとおり future work に属する)。以下の例では、この hcomp 構成子を毎回書かないが、考える各高次帰納型の定義にはつねに含まれているものとする。

Remark 36 の見取り図の第二の場合で触れた「hcomp を残す設計」とは、自然数型のような通常の帰納型にこの扱いを適用することにあたる：hcomp を再帰で消去する代わりに、構成子 $\text{hcomp}_{\mathbb{N}}^i$ を持たせてもよい。この「弱い」形の自然数型は通常のもと同値であり、したがって (univalence により) 等しいことが証明できる。

移送については、Section 3.4 の transp がそのまま用いられる： $\Gamma, i : \mathbb{I} \vdash \vec{u} : \vec{P}$ が与えられたとき、線 $A := D(\vec{u})$ に対する $\text{transp}^i A \varphi u_0$ である。ただしその κ -簡約は、Section 3.4 のような型の形ごとの再帰では与えられず、 u_0 の取りうる形ごとに与えられる：すなわち構成子 c ごとに 1 つ、そして hcomp 構成子に対して 1 つである。後者は

$$\begin{aligned} & \text{transp}^i A \varphi (\text{hcomp}_{A[0/i]}^j [\psi \mapsto u] u_0) \\ & \rightarrow_{\kappa} \text{hcomp}_{A[1/i]}^j [\psi \mapsto \text{transp}^i A \varphi u] (\text{transp}^i A \varphi u_0) \end{aligned}$$

で与えられる (i と j はともに束縛変数なので、名前替えにより $i \neq j$ と仮定してよい)。hcomp の場合はすべての例で同じ形なので、以下では各高次帰納型の transp の定義からは省略する。

Section 3.5 の comp の定義は、hcomp と transp (と squeeze) のみを用いていたことを思い出そう。したがって、高次帰納型の合成は、「構成子に適用された場合の transp」を与えるだけで定まる。

4.2 The Circle

共通パターンの最初の例として、最も単純な高次帰納型である円周 (circle) S^1 を見よう。

Definition 45 (The Circle S^1). 高次帰納型 S^1 は、パラメータなしで、次の2つの構成子により定義される：

$$\frac{\Gamma \text{ context}}{\Gamma \vdash \text{base} : S^1} \quad \frac{\Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \text{loop } r : S^1}$$

ただし loop は次の境界の等値性判断 (∂ -等値性) をともなう：

$$\Gamma \vdash \text{loop } 0 =_{\partial} \text{base} : S^1 \quad \Gamma \vdash \text{loop } 1 =_{\partial} \text{base} : S^1$$

(共通パターンに従い、構成子 $\text{hcomp}_{S^1}^i$ も定義に含まれる。)

base は point 構成子、 loop は path 構成子であり、 $\langle i \rangle \text{loop } i : \text{Path } S^1 \text{ base base}$ を与える。すなわち S^1 は、1つの点と、その点から自分自身への1本の path とからなる型である。名前のとおり、これは円周を表す。

ここで、等号型 (Section 3.7) との違いが鮮明になる。等号型のもとで UIP を仮定すれば、 base から base への証明はすべて $\text{refl}(\text{base})$ に等しいことになり、 loop は 1_{base} と区別できない。しかし Path 型のもとでは両者は区別され、実際 $\text{Path}(\text{Path } S^1 \text{ base base}) 1_{\text{base}} (\langle i \rangle \text{loop } i)$ は inhabited でないことが証明できる^{*2}。これが Section 3.7 で述べた「型は集合ではなく高次の構造をもちうる」ことの最も簡単な実例である。

S^1 はパラメータをもたないから、線が定数の場合しか生じず、 transp の κ -簡約は自明である：

$$\text{transp}^i S^1 \varphi u_0 \rightarrow_{\kappa} u_0$$

したがって S^1 上の合成もすべて homogeneous であり、 hcomp だけで足りる。

型 C への関数 $f : S^1 \rightarrow C$ を定義するには、点 $c : C$ とループ $l : \text{Path } C c c$ を与えればよく、 $f(\text{base}) \rightarrow_{\beta} c$ 、 $f(\text{loop } r) \rightarrow_{\beta} l r$ 、および hcomp との可換性が成り立つ。依存版 (motive $P : S^1 \rightarrow \text{type}$) では、 loop に対する証明義務は P 自身が loop に沿って動くため、Section 2.3 の依存 path

$$\bar{l} : \text{PathP}^i P(\text{loop } i) \bar{c} \bar{c} \quad (\bar{c} : P(\text{base}) \text{ に対して})$$

となる。これが PathP を導入した理由である。

Exercise 46. $\ell \stackrel{\text{def}}{=} \langle i \rangle \text{loop } i$ とおく。本文の recursion principle のデータとして $c \stackrel{\text{def}}{=} \text{base}$ 、 $l \stackrel{\text{def}}{=} \ell \cdot \ell$ (Section 3.3 の合成) を与えると、「2周する」写像 $\text{double} : S^1 \rightarrow S^1$ が定義される。(1) $\ell \cdot \ell : \text{Path } S^1 \text{ base base}$ が導出できること—合成の hcomp の前提 (Lemma 30) が満たされること—を確かめよ。(2) $\text{double}(\text{loop } r)$ が β -規則と $(\text{Path}\beta)$ でどんな項に簡約されるかを書き下せ。それは構成子頭ではなく hcomp 頭の項である。 r に 0, 1 を代入し、 ∂ -等値性によりその値が base に等しいことを検算せよ。(3) 同様に、 $l \stackrel{\text{def}}{=} \ell^{-1} = \langle i \rangle \text{loop } (1-i)$ (Exercise 16) をデータとして「逆回り」の写像を定義し、こちらは $\text{loop } r$ の行き先が構成子頭のままであることを確かめよ。

^{*2} S^1 の基本群が $\mathbb{Z}$ であること ($\pi_1(S^1) = \mathbb{Z}$) の帰結である。homotopy type theory における証明は The Univalent Foundations Program (2013) の第8章を、cubical type theory での構成的な扱いは Mörtberg (2021) を参照。本書では立ち入らない。

4.3 The Torus

Section 4.1 のスキーマは、長さ 2 の名前リストをもつ構成子—square 構成子—を許すが、ここまでの例はいずれも長さ 1 (path 構成子) であった。square 構成子の代表的な実例がトーラス T^2 である。

Definition 47 (The Torus T^2). 高次帰納型 T^2 は、パラメータなしで、次の構成子により定義される ($k = 1, 2$):

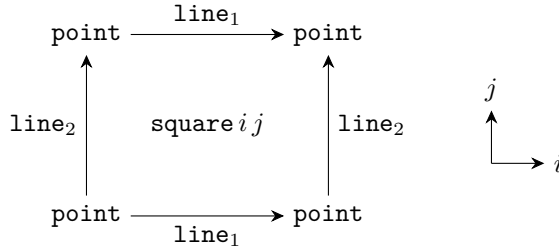
$$\frac{\Gamma \text{ context}}{\Gamma \vdash \text{point} : T^2} \quad \frac{\Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \text{line}_k r : T^2} \quad \frac{\Gamma \vdash r : \mathbb{I} \quad \Gamma \vdash s : \mathbb{I}}{\Gamma \vdash \text{square } rs : T^2}$$

ただし line_k と square は次の境界の等値性判断 (∂ -等値性) をともなう:

$$\begin{aligned} \Gamma \vdash \text{line}_k 0 =_{\partial} \text{point} : T^2 & \quad \Gamma \vdash \text{line}_k 1 =_{\partial} \text{point} : T^2 \\ \Gamma \vdash \text{square } 0s =_{\partial} \text{line}_2 s : T^2 & \quad \Gamma \vdash \text{square } 1s =_{\partial} \text{line}_2 s : T^2 \\ \Gamma \vdash \text{square } r0 =_{\partial} \text{line}_1 r : T^2 & \quad \Gamma \vdash \text{square } r1 =_{\partial} \text{line}_1 r : T^2 \end{aligned}$$

(共通パターンに従い、構成子 $\text{hcomp}_{T^2}^i$ も定義に含まれる。)

square の境界の 4 本の等値性は、対辺を貼り合わせた正方形というトーラスの標準的な描像をそのまま読み上げたものである: 下辺 ($j = 0$) と上辺 ($j = 1$) がともに line_1 、左辺 ($i = 0$) と右辺 ($i = 1$) がともに line_2 である。



同じ型を等号型の語彙で与えると、次のようになる (The Univalent Foundations Program (2013) §6.6):

$$\text{point} : T^2 \quad p, q : \text{point} =_{T^2} \text{point} \quad \text{surf} : p \bullet q = q \bullet p$$

ここで注目すべきは、最後の構成子を述べるために path の合成 $\bullet$ (本書の $p \cdot q$ にあたる、J から導出される演算) が必要になることである。型の定義が導出済みの演算に依存し、しかも $p \bullet q$ という書き方には合成の結合の向きを選択が入り、2 本の辺は非対称に (一方は等式の左辺、他方は右辺に) 現れる。cubical のスキーマでは、 square は単に区間変数を 2 つもつ項であり、その境界を 4 本の ∂ -等値性で対等に読み上げるだけでよい—導出された演算も J も現れない。 n 次元の構成子は n 個の区間変数をもつ項である、という形で、次元が上がっても枠組みは変わらないのである。

この記法上の差は、証明の労力の差として現れる。たとえば $T^2 \simeq S^1 \times S^1$ は、等号型の語彙では独立の論文を要する仕事だったが (Sojakova による 2016 年の ACM TOCL 論文)、cubical type theory では大幅に短い証明が知られている。なお依存除去

では、`square` の像の型が 2 つの区間変数の両方に依存するため、証明義務は依存 `path` (Section 2.3) の 2 次元版—`PathP` の入れ子—となる。また T^2 はパラメータをもたないから、 S^1 と同様、`transp` の κ -簡約は自明であり、 T^2 上の合成はすべて homogeneous である。

Exercise 48. S^1 の場合 (Section 4.2) にならって、型 C への関数 $g : T^2 \rightarrow C$ を定義するために与えるべきデータを構成子ごとに書き出せ。`square` に対して要求されるのは、区間変数を 2 つもち、Definition 47 の 4 本の境界とデータの行き先が整合するような項である。そのうえで、第 1 の射影 $\text{pr}_1 : T^2 \rightarrow S^1$ を、データ

`point` $\mapsto$ `base` `line1` $r \mapsto$ `loop` r `line2` $s \mapsto$ `base` `square` $r\ s \mapsto$ `loop` r

で定義し、`square` に要求される 4 本の境界条件が、`loop` の ∂ -等値性によりすべて満たされることを検算せよ。また、`line2` を `loop` へ、`line1` を定値へ送る第 2 の射影 pr_2 のデータも与えよ。(幾何学的には、トーラスの 2 つの周回方向への射影である。対にした $T^2 \rightarrow S^1 \times S^1$ が同値になることが、先に触れた Sojakova の定理である。)

4.4 The Integers

整数は自然数の対 (a, b) によって「差 $a - b$ 」として表現できる。このとき (a, b) と $(s(a), s(b))$ は同じ整数を表すから、この重複はなんらかの仕方で同一視されなければならない。setoid によるアプローチでは、同値関係を構造として持ち回ることによってこの同一視を実現する。高次帰納型を用いれば、同一視そのものを `path` 構成子として型の定義に組み込むことができる。

Definition 49 (The Integer Type $\mathbb{Z}$). 高次帰納型 $\mathbb{Z}$ は、パラメータなしで、次の 2 つの構成子により定義される：

$$\frac{\Gamma \vdash a : \mathbb{N} \quad \Gamma \vdash b : \mathbb{N}}{\Gamma \vdash \text{diff } a\ b : \mathbb{Z}} \quad \frac{\Gamma \vdash a : \mathbb{N} \quad \Gamma \vdash b : \mathbb{N} \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \text{cancel } a\ b\ r : \mathbb{Z}}$$

ただし `cancel` は次の境界の等値性判断 (Section 3.6 の ∂ -等値性) をともなう：

$$\begin{aligned} \Gamma \vdash \text{cancel } a\ b\ 0 &=_{\partial} \text{diff } a\ b : \mathbb{Z} \\ \Gamma \vdash \text{cancel } a\ b\ 1 &=_{\partial} \text{diff } (s(a))\ (s(b)) : \mathbb{Z} \end{aligned}$$

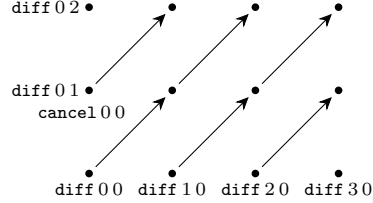
(Section 4.1 の共通パターンに従い、構成子 $\text{hcomp}_{\mathbb{Z}}^i$ も定義に含まれる。)

`diff` は `point` 構成子であり、`diff` $a\ b$ が整数 $a - b$ を表す^{*3}。`cancel` は `path` 構成子であり、

$$\langle i \rangle \text{cancel } a\ b\ i : \text{Path } \mathbb{Z} (\text{diff } a\ b) (\text{diff } (s(a))\ (s(b)))$$

^{*3} 本章の $\mathbb{Z}$ は、生成関係を `cancel` の 1 本に絞る、set truncation を置かない構成である (集合性は後の Theorem 62 で証明する)。対照的に、同値関係 $m + q = p + n$ の全体を `path` 構成子 `eq` とし、さらに truncation 構成子を加える「商型」スタイルの構成もある。後者は筆者らの Cubical Agda チュートリアル <https://github.com/DaisukeBekki/AgdaTutorial-LatticeTheory> (第 IV 部) で機械検証つきで展開されており、そこでは setoid による構成の well-definedness 補題がそのまま `eq` の場合分けに現れる。本章の構成 (`cancel` 版) も同リポジトリの `IntCancel` モジュールで型検査済みである (History and Further Reading 参照)。

を与える。幾何学的には、 $\mathbb{Z}$ は格子 $\mathbb{N} \times \mathbb{N}$ の各点 (a, b) に point $\text{diff } a b$ を置き、対角線方向の辺 $\text{cancel } a b$ で結んだものである（横方向が a 、縦方向が b ）：



各対角線（連結成分）が 1 つの整数に対応する。

$\mathbb{Z}$ はパラメータをもたないから、線が定数の場合しか生じず、 transp の κ -簡約 (Section 4.1) は自明である：

$$\text{transp}^i \mathbb{Z} \varphi u_0 \rightarrow_{\kappa} u_0$$

(Section 4.2 の S^1 と同じ状況である。) それゆえ、以下の $\mathbb{Z}$ 上の構成で用いる合成はすべて homogeneous であり、 hcomp だけで足りる。

型 C への関数 $f : \mathbb{Z} \rightarrow C$ を定義するには (recursion principle)、次のデータを与えればよい：

$$\begin{aligned} d &: (a : \mathbb{N}) \rightarrow (b : \mathbb{N}) \rightarrow C \\ cl &: (a : \mathbb{N}) \rightarrow (b : \mathbb{N}) \rightarrow \text{Path } C (d a b) (d (\mathbf{s}(a)) (\mathbf{s}(b))) \end{aligned}$$

このとき f は次の簡約規則を満たす：

$$\begin{aligned} f(\text{diff } a b) &\rightarrow_{\beta} d a b \\ f(\text{cancel } a b r) &\rightarrow_{\beta} cl a b r \\ f(\text{hcomp}_{\mathbb{Z}}^i [\varphi \mapsto v] v_0) &\rightarrow_{\beta} \text{hcomp}_C^i [\varphi \mapsto f(v)] (f(v_0)) \end{aligned}$$

これらは構成子の直後に除去子 f を適用する detour の除去であるから、Section 3.6 の分類では κ ではなく β である。最後の規則は f が hcomp と可換であることを表す (C が Section 3.3 の型構成子で作られる型の場合、その hcomp は同節の κ -簡約で与えられる)。

以下では、Section 3.3 で定義した path の合成 $p \cdot q$ (hcomp による) と反転 p^{-1} (代入 $[1-i/i]$ による) を用いる。

■ $\mathbb{Z}$ 上の加法 自然数の加法

$$m + n \stackrel{\text{def}}{=} \text{natrec}_m^{\lambda-. \mathbb{N}}(n, \lambda x. \lambda y. \mathbf{s}(y))$$

を用いる。この定義は第 1 引数上の再帰であるから、

$$0 + n = n \quad \mathbf{s}(m) + n = \mathbf{s}(m + n)$$

が判断的等値性として成り立つ。一方、対称的な等式 $m + \mathbf{s}(n) = \mathbf{s}(m + n)$ は判断的には成り立たず、帰納法による証明を要する：

Lemma 50. 任意の $m, n : \mathbb{N}$ に対して、motive を $P \stackrel{\text{def}}{=} \lambda x. \text{Path } \mathbb{N} (x + \mathbf{s}(n)) (\mathbf{s}(x + n))$ として

$$\text{addSucc}(m, n) \stackrel{\text{def}}{=} \text{natrec}_m^P(1_{\mathbf{s}(n)}, \lambda x. \lambda p. \langle i \rangle \mathbf{s}(p i)) : \text{Path } \mathbb{N} (m + \mathbf{s}(n)) (\mathbf{s}(m + n))$$

確認：基底では $0 + \mathbf{s}(n) = \mathbf{s}(n) = \mathbf{s}(0 + n)$ が判断的に成り立つから、定値 $\text{path } 1_{\mathbf{s}(n)}$ でよい。帰納段階では、 $p : \text{Path } \mathbb{N} (x + \mathbf{s}(n)) (\mathbf{s}(x + n))$ から $\langle i \rangle \mathbf{s}(p i)$ が端点 $\mathbf{s}(x + \mathbf{s}(n)) = \mathbf{s}(x) + \mathbf{s}(n)$ と $\mathbf{s}(\mathbf{s}(x + n)) = \mathbf{s}(\mathbf{s}(x) + n)$ をもつ（いずれの等式も判断的）。

$\mathbb{Z}$ 上の加法 $+_{\mathbb{Z}}$ は、第 1 引数・第 2 引数の順に recursion principle を適用して定義する。point 構成子どうしの場合は成分ごとの加法である：

$$\text{diff } a b +_{\mathbb{Z}} \text{diff } c d \stackrel{\text{def}}{=} \text{diff } (a + c) (b + d)$$

第 1 引数が cancel の場合は、判断的等値性 $\mathbf{s}(m) + n = \mathbf{s}(m + n)$ のおかげで、そのまま cancel で応えられる：

$$\text{cancel } a b i +_{\mathbb{Z}} \text{diff } c d \stackrel{\text{def}}{=} \text{cancel } (a + c) (b + d) i$$

実際、右辺の端点は $i = 0$ で $\text{diff } (a + c) (b + d)$ 、 $i = 1$ で $\text{diff } (\mathbf{s}(a + c)) (\mathbf{s}(b + d)) = \text{diff } (\mathbf{s}(a) + c) (\mathbf{s}(b) + d)$ であり、要求される端点と判断的に一致する。

これに対して第 2 引数が cancel の場合、要求される path の端点は $\text{diff } (a + c) (b + d)$ と $\text{diff } (a + \mathbf{s}(c)) (b + \mathbf{s}(d))$ である。後者は $\text{diff } (\mathbf{s}(a + c)) (\mathbf{s}(b + d))$ と判断的には一致せず、Lemma 50 の path で結ばれるにとどまる。ここで hcomp による合成が必要になる： $q_1 \stackrel{\text{def}}{=} \text{addSucc}(a, c)$ 、 $q_2 \stackrel{\text{def}}{=} \text{addSucc}(b, d)$ とおいて

$$\begin{aligned} & \text{diff } a b +_{\mathbb{Z}} \text{cancel } c d i \\ & \stackrel{\text{def}}{=} \text{hcomp}_{\mathbb{Z}}^j [(i = 0) \mapsto \text{diff } (a + c) (b + d), (i = 1) \mapsto \text{diff } (q_1 (1 - j)) (q_2 (1 - j))] \\ & \quad (\text{cancel } (a + c) (b + d) i) \end{aligned}$$

と定義する。この hcomp は次の正方形の破線（蓋）である：

$$\begin{array}{ccc} \text{diff } (a + c) (b + d) & \text{-----} \rightarrow & \text{diff } (a + \mathbf{s}(c)) (b + \mathbf{s}(d)) \\ \uparrow & & \uparrow \\ \text{diff } (a + c) (b + d) & \xrightarrow{\text{cancel } (a + c) (b + d) i} & \text{diff } (\mathbf{s}(a + c)) (\mathbf{s}(b + d)) \end{array} \quad \begin{array}{c} j \\ \uparrow \\ i \end{array}$$

なお、厳密には $+_{\mathbb{Z}}$ の定義は第 1 引数上の recursion で行われ、その target は関数型 $\mathbb{Z} \rightarrow \mathbb{Z}$ である。このとき第 1 引数の $\text{cancel } a b$ に対する証明義務は関数の間の path であり、関数外延性 (Section 2.2) により各点ごとの path を与えればよい。第 2 引数が point 構成子の場合は上の $\text{cancel } (a + c) (b + d)$ の path であり、第 2 引数が $\text{cancel } c d j$ の場合には、上の 2 つの構成 (cancel と hcomp 合成) を対辺にもつ正方形—「対角の正方形」—を与える必要がある。この正方形の境界は、 j 方向が第 2 引数の cancel 、 k 方向が第 1

引数の `cancel` であり、4 隅はいずれも判断的に一致する。その構成は、Corollary 64 ($\mathbb{Z}$ の任意の正方形の境界は充填できる) からただちに得られる。Corollary 64 は $\mathbb{Z}$ が集合であることの帰結であり、その証明 (Theorem 62) は加法を用いないので、循環はない。

■ **$\mathbb{Z}$ の加法の結合律** まず自然数の結合律を `path` として構成する：

Lemma 51 (Associativity of $+$ on $\mathbb{N}$). 任意の $m, n, k : \mathbb{N}$ に対して、motive を $Q \stackrel{\text{def}}{=} \lambda x. \text{Path } \mathbb{N} ((x + n) + k) (x + (n + k))$ として

$$\begin{aligned} \text{assoc}_{\mathbb{N}}(m, n, k) &\stackrel{\text{def}}{=} \text{natrec}_m^Q(1_{n+k}, \lambda x. \lambda p. \langle i \rangle s(p i)) \\ &: \text{Path } \mathbb{N} ((m + n) + k) (m + (n + k)) \end{aligned}$$

確認：基底では $(0 + n) + k = n + k = 0 + (n + k)$ が判断的に成り立つ。帰納段階では $\langle i \rangle s(p i)$ の端点が $s((x + n) + k) = ((s(x) + n) + k)$ と $s(x + (n + k)) = s(x) + (n + k)$ になる (いずれも判断的)。さらに、`natrec` の計算規則により、判断的等値性

$$\text{assoc}_{\mathbb{N}}(s(m), n, k) = \langle i \rangle s(\text{assoc}_{\mathbb{N}}(m, n, k) i)$$

が成り立つことに注意しよう (次の定理の証明で用いる)*4。

Theorem 52 (Associativity of $+$ on $\mathbb{Z}$). 任意の $x, y, w : \mathbb{Z}$ に対して、以下が成り立つ。

$$\vdash \text{Path } \mathbb{Z} ((x +_{\mathbb{Z}} y) +_{\mathbb{Z}} w) (x +_{\mathbb{Z}} (y +_{\mathbb{Z}} w)) \text{ true}$$

証明の構成を、除去規則の証明義務ごととに与える。

(1) x, y, w がすべて point 構成子の場合。 $x = \text{diff } a b$, $y = \text{diff } c d$, $w = \text{diff } e g$ とすると、両辺はそれぞれ $\text{diff}((a + c) + e)((b + d) + g)$ と $\text{diff}(a + (c + e))(b + (d + g))$ に判断的に計算されるから、

$$\langle i \rangle \text{diff}(\text{assoc}_{\mathbb{N}}(a, c, e) i)(\text{assoc}_{\mathbb{N}}(b, d, g) i)$$

が求める `path` である。

(2) $x = \text{cancel } a b k$ の場合 (y, w は point 構成子のまま)。証明義務は、 k を動かしたとき (1) の `path` たちを結ぶ正方形である。記法を簡潔にするため

$$\begin{aligned} A_0 &\stackrel{\text{def}}{=} (a + c) + e & B_0 &\stackrel{\text{def}}{=} (b + d) + g \\ A_1 &\stackrel{\text{def}}{=} a + (c + e) & B_1 &\stackrel{\text{def}}{=} b + (d + g) \\ \alpha &\stackrel{\text{def}}{=} \text{assoc}_{\mathbb{N}}(a, c, e) & \alpha' &\stackrel{\text{def}}{=} \text{assoc}_{\mathbb{N}}(s(a), c, e) \\ \beta &\stackrel{\text{def}}{=} \text{assoc}_{\mathbb{N}}(b, d, g) & \beta' &\stackrel{\text{def}}{=} \text{assoc}_{\mathbb{N}}(s(b), d, g) \end{aligned}$$

*4 この判断的等値性は、`assocN` を「帰納段階で $\langle i \rangle s(p i)$ を返す」形に定義したことに依存している。同じ命題を別の形で証明したもの—たとえば等号型で証明してから Section 3.7 の `toPath` で変換したもの—では、この判断的等値性は失われ、(2) の `naturality` 正方形は直接には通らない。機械検証でも、 $\mathbb{N}$ の補題を `path` としてこの形で証明し直すことが必須であった。`addZero` の同種の等値性についても同様である。

とくと、要求される正方形 $S(k, i)$ の境界は次のとおりである：

$$\begin{aligned} S(k, i)[0/i] &= \text{cancel } A_0 B_0 k && \text{(左辺を判断的に計算した結果)} \\ S(k, i)[1/i] &= \text{cancel } A_1 B_1 k && \text{(右辺を判断的に計算した結果)} \\ S(k, i)[0/k] &= \text{diff } (\alpha i) (\beta i) && \text{((1) の path at } (a, b)) \\ S(k, i)[1/k] &= \text{diff } (\alpha' i) (\beta' i) && \text{((1) の path at } (s(a), s(b))) \end{aligned}$$

この正方形は、hcomp を用いることなく、構成子 `cancel` の引数に `path` を通すだけで直接構成できる：

$$\begin{array}{ccc} & \xrightarrow{\text{cancel } A_1 B_1 k} & \text{diff } (s(A_1)) (s(B_1)) \\ \text{diff } (\alpha i) (\beta i) \uparrow & & \uparrow \text{diff } (s(\alpha i)) (s(\beta i)) \\ & \xrightarrow{\text{cancel } A_0 B_0 k} & \text{diff } (s(A_0)) (s(B_0)) \end{array}$$

$\begin{array}{c} i \\ \uparrow \\ k \end{array}$

実際、面を計算すると： $i = 0$ 面は `cancel  $A_0 B_0 k$` 、 $i = 1$ 面は `cancel  $A_1 B_1 k$` 、 $k = 0$ 面は `diff  $(\alpha i) (\beta i)$` 、 $k = 1$ 面は `diff  $(s(\alpha i)) (s(\beta i))$` である。最後の面が (1) の path at $(s(a), s(b))$ に一致すること、すなわち $\alpha' = \langle i \rangle s(\alpha i)$ および $\beta' = \langle i \rangle s(\beta i)$ は、Lemma 51 の後に注意した判断的等値性から従う。

(3) $y = \text{cancel } c d k$ (または $w = \text{cancel } e g k$) の場合。このときは $+\mathbb{Z}$ の定義の第 2 引数 `cancel` の場合 (hcomp 合成) が現れるため、証明義務の正方形は、その辺自体が hcomp 項であるようなものになる。これを (2) のように直接構成しようとすると、`addSucc` と `assocN` の間の整合性を高次元で追う必要があり、煩雑である。そこで Corollary 64 を用いる： $\mathbb{Z}$ は集合であるから、この正方形の境界も充填でき、証明義務は満たされる。

この証明の構成には、2つのモードが現れることに注意しよう。第一は、(2) のように構成子の引数に `path` や正方形を直接通すことで、証明義務が判断的計算だけで果たされる場合である。第二は、(3) のように判断的に一致しない端点どうしを hcomp による合成・充填で繋ぐ場合である。setoid によるアプローチでは同値関係の保存をすべて命題として持ち回る必要があるのに対し、高次帰納型では多くの整合性が第一のモードにより判断的等値性として無償で得られ、残りが第二のモードの明示的構成となる。

4.5 Basic Theorems on $\mathbb{Z}$

本節では、setoid や古典的な商集合による構成で示される $\mathbb{Z}$ の基本定理たちを、Path 型の構成として証明する。どの定理で hcomp が必要になり、どの定理では不要か (判断的計算・構成子への `path` の直接適用で足りるか) に注目してほしい。

まず、 $\mathbb{N}$ 上の補題を 2 つ用意する (Lemma 50 の続きである)。以下、`path` の合成・と反転 $(-)^{-1}$ は Section 3.3 のものであり、 $\mathbb{N}$ 上の hcomp は Section 3.3 の再帰的定義 (または Section 4.1 の hcomp 構成子) による。

Lemma 53 (Lemmas on $\mathbb{N}$). 任意の $m, n : \mathbb{N}$ に対して、次の 2 つの path が構成できる。motive を $P_1 \stackrel{\text{def}}{=} \lambda x. \text{Path } \mathbb{N} (x+0) x$, $P_2 \stackrel{\text{def}}{=} \lambda x. \text{Path } \mathbb{N} (x+n) (n+x)$ とし

$$\begin{aligned} \text{addZero}(m) &\stackrel{\text{def}}{=} \text{natrec}_{P_1}^m(1_0, \lambda x. \lambda p. \langle i \rangle s(p i)) : \text{Path } \mathbb{N} (m+0) m \\ \text{comm}_{\mathbb{N}}(m, n) &\stackrel{\text{def}}{=} \text{natrec}_{P_2}^m(\text{addZero}(n)^{-1}, \lambda x. \lambda p. (\langle i \rangle s(p i)) \cdot \text{addSucc}(n, x)^{-1}) \\ &\quad : \text{Path } \mathbb{N} (m+n) (n+m) \end{aligned}$$

確認: $\text{comm}_{\mathbb{N}}$ の基底は $\text{Path } \mathbb{N} (0+n) (n+0) = \text{Path } \mathbb{N} n (n+0)$ だから $\text{addZero}(n)^{-1}$ でよい。帰納段階では $s(x) + n = s(x+n)$ (判断的) から出発して $\langle i \rangle s(p i)$ で $s(n+x)$ に至り、 $\text{addSucc}(n, x)^{-1}$ で $n + s(x)$ に至る。また natrec の計算規則により $\text{addZero}(s(m)) = \langle i \rangle s(\text{addZero}(m) i)$ が判断的に成り立つ (後で用いる)。

Lemma 54 (Equivalent Differences). 任意の $k, a, b : \mathbb{N}$ に対して、次の 2 つの path が構成できる。motive を $Q_1 \stackrel{\text{def}}{=} \lambda x. \text{Path } \mathbb{Z} (\text{diff } a b) (\text{diff } (x+a) (x+b))$, $Q_2 \stackrel{\text{def}}{=} \lambda x. \text{Path } \mathbb{Z} (\text{diff } x x) (\text{diff } 0 0)$ とし

$$\begin{aligned} \text{shift}(k, a, b) &\stackrel{\text{def}}{=} \text{natrec}_{Q_1}^k(1_{\text{diff } a b}, \lambda x. \lambda p. p \cdot \langle i \rangle \text{cancel } (x+a) (x+b) i) \\ &\quad : \text{Path } \mathbb{Z} (\text{diff } a b) (\text{diff } (k+a) (k+b)) \\ \text{diffZero}(k) &\stackrel{\text{def}}{=} \text{natrec}_{Q_2}^k(1_{\text{diff } 0 0}, \lambda x. \lambda p. (\langle i \rangle \text{cancel } x x (1-i)) \cdot p) \\ &\quad : \text{Path } \mathbb{Z} (\text{diff } k k) (\text{diff } 0 0) \end{aligned}$$

shift の帰納段階では、IH の path p を、端点が判断的に合う ($s(x) + a = s(x+a)$) path 構成子 $\text{cancel } (x+a) (x+b)$ と hcomp で合成している。 diffZero の帰納段階は、 cancel の「逆向き」の path (代入 $[1-i/i]$ による反転) と IH の合成である。たとえば $\text{shift}(8, 7, 2) : \text{Path } \mathbb{Z} (\text{diff } 7 2) (\text{diff } 15 10)$ は、古典的構成でいう「差分 $\langle 7, 2 \rangle$ と $\langle 15, 10 \rangle$ は同値である」ことの witness にあたる。

Definition 55 (Negation and Subtraction on $\mathbb{Z}$). $\text{neg} : \mathbb{Z} \rightarrow \mathbb{Z}$ を recursion principle により次のデータで定義する：

$$\text{neg}(\text{diff } a b) \stackrel{\text{def}}{=} \text{diff } b a \quad \text{neg}(\text{cancel } a b i) \stackrel{\text{def}}{=} \text{cancel } b a i$$

また、減法を $y -_{\mathbb{Z}} x \stackrel{\text{def}}{=} y +_{\mathbb{Z}} \text{neg}(x)$ と定義する。

cancel の場合のデータ $\langle i \rangle \text{cancel } b a i$ は、端点が $\text{diff } b a$ と $\text{diff } (s(b)) (s(a))$ であり、要求される端点 (neg の point での値) と判断的に一致する。すなわち neg は「構成子を構成子に写す」だけの写像であり、 hcomp を一切必要としない。古典的構成では $-[m, n]_i = [n, m]_i$ は証明すべき命題であったが、ここではそれが定義そのものになっている。

Theorem 56 (Double Negation). 任意の $x : \mathbb{Z}$ に対して、以下が成り立つ。

$$\vdash \text{Path } \mathbb{Z} (\text{neg}(\text{neg}(x))) x \text{ true}$$

証明:motive $\lambda x. \text{Path } \mathbb{Z} (\text{neg}(\text{neg}(x))) x$ による除去。point の場合は $\text{neg}(\text{neg}(\text{diff } a b)) = \text{diff } a b$ が判断的に成り立つから $1_{\text{diff } a b}$ でよい。cancel $a b k$ の場合の証明義務の正方形は、 $\text{neg}(\text{neg}(\text{cancel } a b k)) = \text{cancel } a b k$ (判断的) により退化しており、 k 方向にのみ動く定値の正方形 $S(k, i) \stackrel{\text{def}}{=} \text{cancel } a b k$ で満たされる。hcomp の場合は、除去が hcomp と可換であることから機械的に処理される (以後、この場合分けは省略する)。このように、この定理は hcomp を全く必要としない: Theorem 52 (結合律) との対比に注意してほしい。

次の定理では、Section 3.3 で導入した hcomp の充填 hfill (開いた箱の底面と蓋を結ぶ内部) を用いる。以下、 $0_{\mathbb{Z}} \stackrel{\text{def}}{=} \text{diff } 0 0$ とおく。

Theorem 57 (Unit Laws). 任意の $x : \mathbb{Z}$ に対して、以下がともに成り立つ。

$$\vdash \text{Path } \mathbb{Z} (0_{\mathbb{Z}} +_{\mathbb{Z}} x) x \text{ true}$$

$$\vdash \text{Path } \mathbb{Z} (x +_{\mathbb{Z}} 0_{\mathbb{Z}}) x \text{ true}$$

証明 (左単位元律): point の場合、 $0_{\mathbb{Z}} +_{\mathbb{Z}} \text{diff } c d = \text{diff } (0 + c) (0 + d) = \text{diff } c d$ が判断的に成り立つから、定値 $\text{path } 1_{\text{diff } c d}$ でよい。cancel $c d k$ の場合、 $+_{\mathbb{Z}}$ の定義 (第 2 引数 cancel の場合) に現れる $q_1 = \text{addSucc}(0, c)$ は addSucc の基底ゆえ判断的に定値 path であり、

$$\begin{aligned} & 0_{\mathbb{Z}} +_{\mathbb{Z}} \text{cancel } c d k \\ &= \text{hcomp}_{\mathbb{Z}}^j [(k = 0) \mapsto \text{diff } c d, (k = 1) \mapsto \text{diff } (s(c)) (s(d))] (\text{cancel } c d k) \end{aligned}$$

と、側面が定値の hcomp になる。証明義務は、この hcomp と底面 cancel $c d k$ を結ぶ正方形であり、まさに hfill が与えるものである:

$$\begin{aligned} & Q(k, i) \\ & \stackrel{\text{def}}{=} \text{hfill}_{\mathbb{Z}}^j [(k = 0) \mapsto \text{diff } c d, (k = 1) \mapsto \text{diff } (s(c)) (s(d))] (\text{cancel } c d k) (1-i) \end{aligned}$$

($i = 0$ 面が上の hcomp、 $i = 1$ 面が cancel $c d k$ 、 $k = 0$, $k = 1$ 面は定値である。)

証明 (右単位元律): point の場合、 $\text{diff } a b +_{\mathbb{Z}} 0_{\mathbb{Z}} = \text{diff } (a + 0) (b + 0)$ であり、 $a + 0 = a$ は判断的には成り立たないから、Lemma 53 の addZero を用いて

$$\langle i \rangle \text{diff } (\text{addZero}(a) i) (\text{addZero}(b) i) : \text{Path } \mathbb{Z} (\text{diff } (a + 0) (b + 0)) (\text{diff } a b)$$

とする。cancel $a b k$ の場合は、Theorem 52 の (2) と同じ naturality の手筋で、正方形 $S(k, i) \stackrel{\text{def}}{=} \text{cancel } (\text{addZero}(a) i) (\text{addZero}(b) i) k$ が直接効く ($k=1$ 面の一致は、判断的等値性 $\text{addZero}(s(a)) = \langle i \rangle s(\text{addZero}(a) i)$ による)。

左単位元律は「hfill による退化 hcomp の除去」、右単位元律は「 $\mathbb{N}$ の補題の ap と naturality 正方形」と、同じ定理の左右で必要な道具が異なる。これは加法の定義 (第 1 引数上の再帰) に由来する判断的/命題的の非対称性 (Lemma 50 の前後の議論) の帰結である。

Theorem 58 (Additive Inverse). 任意の $x : \mathbb{Z}$ に対して、以下が成り立つ。

$$\vdash \text{Path } \mathbb{Z} (x +_{\mathbb{Z}} \text{neg}(x)) 0_{\mathbb{Z}} \text{ true}$$

証明の構成：point の場合、 $\text{diff } a b +_{\mathbb{Z}} \text{neg}(\text{diff } a b) = \text{diff } (a + b) (b + a)$ であり、

$$(\langle i \rangle \text{diff } (\text{comm}_{\mathbb{N}}(a, b) i) (b + a)) \cdot \text{diffZero}(b + a) : \text{Path } \mathbb{Z} (\text{diff } (a + b) (b + a)) 0_{\mathbb{Z}}$$

が求める path である ($\text{comm}_{\mathbb{N}}$ で第 1 成分を $b + a$ に揃え、 diffZero で対角を $0_{\mathbb{Z}}$ に潰す)。 $\text{cancel } a b k$ の場合は、 x と $\text{neg}(x)$ が同時に動くため、 $+_{\mathbb{Z}}$ の第 1・第 2 引数とともに cancel である「対角」の正方形が現れる。これも Theorem 52 の (3) と同様、Corollary 64 により充填できる。

Exercise 59. $m, n, p, q : \mathbb{N}$ と $\text{path } e : \text{Path } \mathbb{N} (m + q) (p + n)$ が与えられたとき、 $\text{Path } \mathbb{Z} (\text{diff } m n) (\text{diff } p q)$ を構成せよ。(ヒント：shift により $\text{diff } m n$ と $\text{diff } (q + m) (q + n)$ 、 $\text{diff } p q$ と $\text{diff } (n + p) (n + q)$ をそれぞれ結び、 $\text{comm}_{\mathbb{N}}$ と e で成分を揃えよ。)

Exercise 60. 加法交換律 $\text{Path } \mathbb{Z} (x +_{\mathbb{Z}} y) (y +_{\mathbb{Z}} x)$ について、point の場合 ($\text{comm}_{\mathbb{N}}$ の成分ごとの ap) と $x = \text{cancel}$ の場合 (naturality 正方形) を構成せよ。($y = \text{cancel}$ の場合は Theorem 52 の (3) と同種であり、Corollary 64 により充填できる。)

Exercise 61. 任意の $a, b : \mathbb{Z}$ に対して、 $x \stackrel{\text{def}}{=} a -_{\mathbb{Z}} b$ が方程式 $x +_{\mathbb{Z}} b = a$ の解であること、すなわち $\text{Path } \mathbb{Z} ((a -_{\mathbb{Z}} b) +_{\mathbb{Z}} b) a$ を、Theorem 52・Theorem 57・Theorem 58・Exercise 60 を組み合わせた path の合成のみで構成せよ。さらに、解の一意性 ($x +_{\mathbb{Z}} b = a$ を満たす x どうしを結ぶ path) も同様に構成できることを確認せよ。古典的証明 (群論的な等式変形) がそのまま path の合成に翻訳されることに注意。

4.6 The Normal Form Theorem

本節の内容は、本章でここまで定義した加法を一切用いない。逆に、本節で得られる「 $\mathbb{Z}$ は集合である」という結論が、加法の定義と諸法則の証明に残されていた正方形の証明義務をすべて解消する。したがって論理的な順序は、正規形定理が先、加法が後である (加法を先に扱ったのは、hcomp の使い方を見るためであった)。

古典的構成では、任意の整数が $[\langle n, 0 \rangle]_i$ か $[\langle 0, m \rangle]_i$ ($m \neq 0$) のいずれか一方の標準形で表されること、および $\mathbb{N}$ と正の整数と 0 の集合との間の一対一対応が示された。cubical な構成におけるその対応物は、 $\mathbb{Z}$ と「二分表現」の型との同値である。標準形の型を選言型により

$$\text{Int} \stackrel{\text{def}}{=} \mathbb{N} + \mathbb{N}$$

と定め、 $\iota_1(n)$ を整数 n 、 $\iota_2(m)$ を整数 $-(m + 1)$ (すなわち $-1, -2, \dots$) と読む。この二分が古典版の標準形の二分に対応する。

正規化関数 $\text{norm} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Int}$ を、第 2 引数上の再帰により、 $\mathbb{N} \rightarrow \text{Int}$ を target として定義する：

$$\begin{aligned} \text{norm}_0 &\stackrel{\text{def}}{=} \lambda a. \iota_1(a) & \text{norm}(a, b) &\stackrel{\text{def}}{=} \text{norm}_b(a) \\ \text{norm}_{s(b)} &\stackrel{\text{def}}{=} \lambda a. \text{natrec}_{a _ \text{Int}}^{\lambda _ . \text{Int}}(\iota_2(b), \lambda a'. \lambda _ . \text{norm}_b(a')) \end{aligned}$$

この符号化により、次の 3 つの等式がいずれも判断的に成り立つ (a が変数のままでも成り立つことが要点である) :

$$\text{norm}(a, 0) = \iota_1(a) \quad \text{norm}(0, \mathbf{s}(b)) = \iota_2(b) \quad \text{norm}(\mathbf{s}(a), \mathbf{s}(b)) = \text{norm}(a, b)$$

第 1 の等式は norm_0 の定義そのものであり、第 2・第 3 の等式は内側の natrec の計算規則による。これを用いて、往復の写像を定義する :

$$\begin{aligned} \text{toInt} : \mathbb{Z} &\rightarrow \text{Int} & \text{toInt}(\text{diff } a \ b) &\stackrel{\text{def}}{=} \text{norm}(a, b) \\ \text{fromInt} : \text{Int} &\rightarrow \mathbb{Z} & \text{fromInt}(\iota_1(n)) &\stackrel{\text{def}}{=} \text{diff } n \ 0 \\ & & \text{fromInt}(\iota_2(m)) &\stackrel{\text{def}}{=} \text{diff } 0 \ (\mathbf{s}(m)) \end{aligned}$$

toInt は $\mathbb{Z}$ の recursion principle による。 $\text{cancel } a \ b$ の場合の証明義務は $\text{norm}(a, b)$ と $\text{norm}(\mathbf{s}(a), \mathbf{s}(b))$ を結ぶ path であるが、上の第 3 の等式が判断的であるから、定値 path

$$cl_{\text{toInt}} \ a \ b \stackrel{\text{def}}{=} 1_{\text{norm}(a, b)}$$

でよい。したがって

$$\text{toInt}(\text{cancel } a \ b \ i) = \text{norm}(a, b) \quad (i \text{ に依存しない})$$

が成り立つ。この事実が (ii) の証明で本質的に効く。(hcomp の場合は Section 3.3 で与えた $\text{Int} = \mathbb{N} + \mathbb{N}$ の κ -簡約による。)

Theorem 62 (Normal Form). toInt と fromInt は互いに逆であり、 $\mathbb{Z} \simeq \text{Int}$ が成り立つ。すなわち、任意の $v : \text{Int}, z : \mathbb{Z}$ に対して、以下がともに成り立つ。

$$\begin{aligned} &\vdash \text{Path Int } (\text{toInt}(\text{fromInt}(v))) \ v \ \text{true} \\ &\vdash \text{Path } \mathbb{Z} \ (\text{fromInt}(\text{toInt}(z))) \ z \ \text{true} \end{aligned}$$

証明 :

(i) $\text{toInt} \circ \text{fromInt}$ の側は Int 上の場合分けによる。 $v = \iota_1(n)$ のとき $\text{toInt}(\text{fromInt}(\iota_1(n))) = \text{toInt}(\text{diff } n \ 0) = \text{norm}(n, 0) = \iota_1(n)$ 、 $v = \iota_2(m)$ のとき $\text{toInt}(\text{fromInt}(\iota_2(m))) = \text{norm}(0, \mathbf{s}(m)) = \iota_2(m)$ がいずれも判断的に成り立つ (norm の第 1・第 2 の等式)。よって定値 path でよい。

(ii) $\text{fromInt} \circ \text{toInt}$ の側は $\mathbb{Z}$ の依存除去による。motive を次のようにとる :

$$M \stackrel{\text{def}}{=} \lambda z. \text{Path } \mathbb{Z} \ (\text{fromInt}(\text{toInt}(z))) \ z$$

point の場合。 $m_{a,b} : \text{Path } \mathbb{Z} \ (\text{fromInt}(\text{norm}(a, b))) \ (\text{diff } a \ b)$ を、 b そして a の二重帰納法で構成する :

$$\begin{aligned} m_{a,0} &\stackrel{\text{def}}{=} 1_{\text{diff } a \ 0} & (\text{fromInt}(\iota_1(a)) = \text{diff } a \ 0 \text{ より}) \\ m_{0,\mathbf{s}(b)} &\stackrel{\text{def}}{=} 1_{\text{diff } 0 \ (\mathbf{s}(b))} & (\text{fromInt}(\iota_2(b)) = \text{diff } 0 \ (\mathbf{s}(b)) \text{ より}) \\ m_{\mathbf{s}(a),\mathbf{s}(b)} &\stackrel{\text{def}}{=} m_{a,b} \cdot \langle i \rangle \text{cancel } a \ b \ i & (\text{norm}(\mathbf{s}(a), \mathbf{s}(b)) = \text{norm}(a, b) \text{ より}) \end{aligned}$$

第3の場合、左辺の型は $\text{Path } \mathbb{Z} (\text{fromInt}(\text{norm}(a, b))) (\text{diff } (s(a)) (s(b)))$ であり、 $m_{a,b}$ の終点 $\text{diff } a b$ から $\langle i \rangle \text{cancel } a b i$ が $\text{diff } (s(a)) (s(b))$ へ至るので、合成の型が合う。

cancel の場合。 証明義務は、 $M(\text{cancel } a b i)$ に沿った依存 path (Section 2.3)、すなわち $m_{a,b}$ と $m_{s(a),s(b)}$ を結ぶ正方形である。先に注意したとおり $\text{toInt}(\text{cancel } a b i) = \text{norm}(a, b)$ は i に依存しないから、

$$M(\text{cancel } a b i) = \text{Path } \mathbb{Z} (\text{fromInt}(\text{norm}(a, b))) (\text{cancel } a b i)$$

となり、 i に沿って動くのは path の終点だけである。よって求める正方形 $S(i, k)$ の境界は

$$\begin{aligned} S(i, k)[0/i] &= m_{a,b} k & S(i, k)[1/i] &= m_{s(a),s(b)} k \\ S(i, k)[0/k] &= \text{fromInt}(\text{norm}(a, b)) \text{ (定値)} & S(i, k)[1/k] &= \text{cancel } a b i \end{aligned}$$

である。ところがこれは、 $m_{s(a),s(b)}$ を定義した合成 $m_{a,b} \cdot \langle i \rangle \text{cancel } a b i$ の充填に他ならない。実際、Section 3.3 の hfill を $p \stackrel{\text{def}}{=} m_{a,b}, q \stackrel{\text{def}}{=} \langle i \rangle \text{cancel } a b i$ に対して用いると、

$$S(i, k) \stackrel{\text{def}}{=} \text{hfill}_{\mathbb{Z}}^n [(k=0) \mapsto \text{fromInt}(\text{norm}(a, b)), (k=1) \mapsto q n] (p k) i$$

が求める正方形を与える。その4面は hfill の性質 (Exercise 33) から直ちに読み取れる： $i=0$ で底面 $p k = m_{a,b} k$ 、 $i=1$ で蓋 $(p \cdot q) k = m_{s(a),s(b)} k$ 、 $k=0$ で側面の定値 $\text{fromInt}(\text{norm}(a, b))$ 、 $k=1$ で側面 $q i = \text{cancel } a b i$ である。

hcomp の場合。 除去が hcomp と可換であることから機械的に処理される。 $\square$

(ii) の **cancel** の場合が hfill そのものであったことは偶然ではない。point の場合の帰納段階で「IH に **cancel** を継ぎ足す」という構成を採ったため、**cancel** に沿った整合性は、その継ぎ足しの過程（すなわち充填）がそのまま与えてくれる。 hcomp で構成したものの整合性は hfill が保証する、という関係は、以下でも繰り返し用いられる。

古典版の定理の証明が $g \circ f = \text{id}$ と $f \circ g = \text{id}$ の2つの計算からなっていたことを思い出そう。cubical 版でも骨格は同じであり、(i) の側は判断的計算だけで済む一方、(ii) の側にはのみ **cancel** の path が必要になる。これは古典版で $\sim_d$ の計算を要したのが $f \circ g$ の側だけであったことに正確に対応している。

この定理からは次が従う：

- $\mathbb{N}$ の等値性が決定可能であることから Int の等値性も決定可能であり、同値 $\mathbb{Z} \simeq \text{Int}$ によりこれを $\mathbb{Z}$ へ移送できる。さらに Hedberg の定理（等値性が決定可能な型は集合である）を適用すると、 $\mathbb{Z}$ は集合 (set) である。すなわち Definition 49 に truncation 構成子を入れなかったことが正当化される。なお、決定可能性から集合性への含意は自明ではなく、Hedberg の定理を要する (The Univalent Foundations Program 2013 の定理 7.2.5 を参照)*5。
- 任意の $z : \mathbb{Z}$ は $\text{diff } n 0$ か $\text{diff } 0 (s(m))$ のいずれか一方と path で結ばれる。これが古典版の「 $\mathbb{Z}$ を二分する」(標準形の存在と一意性) に対応する。正負と0による三分法は、これに Int 側で $l_1(0)$ を分離して得られる。
- 埋め込み $n \mapsto \text{diff } n 0$ は単射である。

*5 $\mathbb{Z}$ の段では Hedberg を経由しない別の道もある：集合性はレトラクトに沿って直接移送できる ($p, q : \text{Path } \mathbb{Z} x y$ が与えられたとき、 toInt による像を Int の集合性で結び、往復の path で $\mathbb{Z}$ へ引き戻せばよい)。この道では Hedberg の定理は Int (実質は $\mathbb{N}$) の集合性にしか使われない。本章の機械検証 (History and Further Reading 参照) はこちらの道を採用している。

このうち第1の帰結は、本章でここまで先送りにしてきた正方形の証明義務をすべて解消する。そのことを述べておこう。

Lemma 63 (Squares in a Set). 型 A が集合であるとき、 A 中の任意の正方形の境界—すなわち4つの path であって、隣り合うものの端点が一致しているもの—は充填できる。

確認：集合であるとは、任意の $x, y : A$ と任意の $p, q : \text{Path } A x y$ に対して $\text{Path } (\text{Path } A x y) p q$ が inhabited であることをいう。正方形の境界が与えられたとき、そのうち3辺を hcomp で合成して4辺目と同じ端点をもつ path を作り、集合性によりこの2つの path を結ぶ path (すなわち正方形) を得て、これと合成の充填 (hfill) とを貼り合わせればよい。

Corollary 64. $\mathbb{Z}$ 中の任意の正方形の境界は充填できる。

(Theorem 62 の第1の帰結と Lemma 63 による。)

したがって、本章で「3次元の開いた箱の充填により構成される」として先送りにした証明義務— $\mathbb{Z}$ の定義における対角 (第1引数・第2引数がともに cancel) の正方形、Theorem 52 の (3)、Theorem 58 の cancel の場合、および Exercise 60 の挑戦問題—は、いずれも $\mathbb{Z}$ の正方形の充填問題であるから、Corollary 64 によりただちに解決する。本節が加法を用いないことを冒頭で確認したのは、この論法に循環がないことを示すためであった。

Remark 65. この筋道は、高次帰納型を扱う際の一般的な戦術である。高次元の整合性 (正方形・立方体……) をすべて手で構成するのは煩雑であり、しばしば非現実的である。そこで、まず対象の型が集合であることを—多くの場合、決定可能な等値性をもつ既知の型との同値を経由して—示し、以後の整合性の証明義務を集合性に委ねる。setoid によるアプローチが、同値関係の保存を最初から命題として持ち回ると比べてほしい。高次帰納型では、まず高次構造を持ちうる型として定義し、必要に応じて後から「実は集合であった」ことを証明する、という順序をとる。なお、最初から集合であることを保証したい場合には、set truncation の構成子 (任意の $p, q : \text{Path } A x y$ に対して $\text{Path } (\text{Path } A x y) p q$ の元を与える構成子) を定義に加える方法もある。Definition 49 でそれを採らなかったのは、Theorem 62 により集合性が導けるからである。

Remark 66. 古典的構成の続きにあたる正値性・順序 ($\mathbb{Z}^+$, $<$, 三分法の述語版、全順序性・整列性) を $\mathbb{Z}$ 上で直接定義しようとする、述語 $P : \mathbb{Z} \rightarrow \text{type}_i$ を除去で定義する際、 cancel の場合に「型と型を結ぶ path 」、すなわち universe への除去が必要になる。これには次章で導入する univalence (ua) が要る。直接の道は Chapter 5 の末尾で、「零である」という述語を例に実演する。代替として、Theorem 62 の同値を経由して Int 側で順序を定義し移送する道もある。

History and Further Reading

高次帰納型は homotopy type theory の文脈で導入され (circle・suspension・truncation・商型などの例とともに HoTT book (The Univalent Foundations Program, 2013) の第6章で展開された)、その意味論は Lumsdaine and Shulman (2020) により与えられた。cubical type theory における高次帰納型の構成的な扱いが Coquand et al. (2018) (本章前半の転記元) であり、合成を hcomp と trans (本書の transp) に分解するその手法は、cartesian 側では Cavallo and Harper (2019) が対応する。

本章の整数型のように、商を高次帰納型として表現する試みには複数の表現が知られている。Altenkirch and Scoccola (2020) は successor と predecessor の同型を構成子として持つ別の表現を与え、truncation なしで set となることの証明を含めて詳述している。本章の `diff` / `cancel` による表現は、古典的な差分による構成をそのまま高次帰納型化したものである。

実装としては、Cohen et al. (2018) に付随する実験的実装 `cubicaltt` に始まり、現在では Agda の `cubical` モード (Vezzosi et al., 2019) が広く用いられている。本章の構成を機械検証するには `Cubical Agda` とその標準ライブラリ (整数型の複数の実装の比較を含む) が適している。実際、本章の諸構成 ($\mathbb{Z}$ とその定理群・正規形定理) と Chapter 5 の `ua` による型値再帰は、講義用リポジトリ <https://github.com/DaisukeBekki/AgdaTutorial-LatticeTheory> の `IntCancel` モジュールにおいて `Cubical Agda` で型検査済みである。とくに、本書が「判断的に成り立つ」と主張した等値性 (`norm` の 3 等式・`toInt` の `cancel` 節の定数性・正規形定理 (ii) の充填・`naturality` 正方形・`ua` の端点) は、いずれも開いた変数のまま `refl` として型検査を通過することが確認された。同リポジトリの第 IV 部は、整数を同値関係全体の `path` 構成子と `set truncation` で与える商型スタイルを採っており、本章の `cancel` 生成・`truncation` なしの構成と相補的である (Definition 49 の脚注を参照)。

Bibliography

- Altenkirch, T., C. McBride, and W. Swierstra. (2007) “Observational equality, now!”, In *Proceedings of the 2007 Workshop on Programming Languages meets Program Verification (PLPV '07)*, A. Stump and H. Xi (eds.). pp.57–68.
- Altenkirch, T. and L. Scoccola. (2020) “The Integers as a Higher Inductive Type”, In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.67–73, ACM.
- Angiuli, C., G. Brunerie, T. Coquand, R. Harper, K.-B. Hou (Favonia), and D. R. Licata. (2021) “Syntax and models of Cartesian cubical type theory”, *Mathematical Structures in Computer Science* **31**(4), pp.424–468.
- Angiuli, C., R. Harper, and T. Wilson. (2017) “Computational higher-dimensional type theory”, In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. pp.680–693, ACM.
- Awodey, S. and M. A. Warren. (2009) “Homotopy theoretic models of identity types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **146**(1), pp.45–55.
- Bekki, D. (2014) “Representing Anaphora with Dependent Types”, In *Proceedings of Logical Aspects of Computational Linguistics (8th international conference, LACL2014, Toulouse, France), LNCS 8535*, N. Asher and S. V. Soloviev (eds.). pp.14–29, Springer, Heidelberg.
- Bekki, D. and K. Mineshima. (2017) “Context-Passing and Underspecification in Dependent Type Semantics”, In: S. Chatzikyriakidis and Z. Luo (eds.): *Modern Perspectives in Type-Theoretical Semantics*, Vol. 98 of *Studies in Linguistics and Philosophy*. Springer, pp.11–41.
- Bezem, M., T. Coquand, and S. Huber. (2014) “A Model of Type Theory in Cubical Sets”, In *Proceedings of 19th International Conference on Types for Proofs and Programs (TYPES 2013)*, R. Matthes and A. Schubert (eds.), Vol. 26 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.107–128, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- Cavallo, E. and R. Harper. (2019) “Higher Inductive Types in Cubical Computational Type Theory”, *Proceedings of the ACM on Programming Languages* **3**(POPL), pp.1:1–1:27.
- Cohen, C., T. Coquand, S. Huber, and A. Mörtberg. (2018) “Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom”, In *Proceedings of 21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (ed.), Vol. 69 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.5:1–5:34, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:1611.02108.
- Coquand, T., S. Huber, and A. Mörtberg. (2018) “On Higher Inductive Types in Cubical Type Theory”, In *Proceedings of LICS '18: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.255–264.

- Hofmann, M. (1995) “Extensional concepts in intensional type theory”, Ph.D. thesis, University of Edinburgh.
- Hofmann, M. and T. Streicher. (1998) “The groupoid interpretation of type theory”, In: G. Sambin and J. M. Smith (eds.): *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*, Oxford Logic Guides, vol.36. New York, Oxford University Press.
- Huber, S. (2019) “Canonicity for Cubical Type Theory”, *Journal of Automated Reasoning* **63**(2), pp.173–210.
- Kapulkin, K. and P. L. Lumsdaine. (2021) “The simplicial model of Univalent Foundations (after Voevodsky)”, *Journal of the European Mathematical Society* **23**(6), pp.2071–2126.
- Lumsdaine, P. L. and M. Shulman. (2020) “Semantics of higher inductive types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **169**(1), pp.159–208.
- Martin-Löf, P. (1975) “An intuitionistic theory of types”, In: H. E. Rose and J. Shepherdson (eds.): *Logic Colloquium ’73*. Amsterdam, North-Holland, pp.73–118.
- Martin-Löf, P. (1984) *Intuitionistic Type Theory*, Vol. 17. Naples, Italy: Bibliopolis. Sambin, Giovanni (ed.).
- Mörtberg, A. (2021) “Cubical methods in homotopy type theory and univalent foundations”, *Mathematical Structures in Computer Science* **31**(10), pp.1147–1184.
- Orton, I. and A. M. Pitts. (2016) “Axioms for Modelling Cubical Type Theory in a Topos”, In *Proceedings of 25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, J.-M. Talbot and L. Regnier (eds.), Vol. 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.24:1–24:19, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Journal version: *Logical Methods in Computer Science* 14(4:23), 2018.
- Sterling, J. and C. Angiuli. (2021) “Normalization for Cubical Type Theory”, In *Proceedings of 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*. pp.1–15, IEEE.
- Streicher, T. (1993) “Investigations into Intensional Type Theory”, Ph.D. thesis, Habilitationsschrift, Ludwig-Maximilians-Universität.
- The Univalent Foundations Program. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*. <https://homotopytypetheory.org/book>.
- Vezzosi, A., A. Mörtberg, and A. Abel. (2019) “Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types”, *Proceedings of the ACM on Programming Languages* **3**, pp.87:1–87:29.